A Wolf in Lamb's Clothing: Computer Science in a Mathematics Course

Michelle Friend
University of Nebraska at Omaha
Omaha, NE
mefriend@unomaha.edu

Andrew W. Swift
University of Nebraska at Omaha
Omaha, NE
aswift@unomaha.edu

Betty Love
University of Nebraska at Omaha
Omaha, NE
blove@unomaha.edu

Victor Winter
University of Nebraska at Omaha
Omaha, NE
vwinter@unomaha.edu

ABSTRACT

If computer science programs face a challenge of convincing students that programming is fun and achievable, they have nothing on mathematics departments who face societal beliefs that math is hard and scary. Several movements in computer science education have focused on broadening participation within computer science and across disciplines. The "CS + X" efforts have focused on helping computer science integrate into other disciplines. The "CS For All" movement has highlighted the importance of providing high quality computing education for all students. Simultaneously, there is increasing attention to the need to provide general education alternatives to college algebra. This paper describes a course designed to combine these goals: a course that uses programming to introduce students to functions, patterns, and spatial and computational thinking in order to meet quantitative reasoning goals set by the university. The course initially used Bricklayer as the programming environment, then transitioned to Processing. Students were successful in writing programs that created art, demonstrated mastery of quantitative literacy, and had improved attitudes following the course. This project suggests that in addition to the creation of introductory computer science classes, courses which embed computer science into disciplinary requirements can be a successful pathway to expand opportunities for students to learn computing.

CCS CONCEPTS

• **Social and professional topics** → **Computing education**; **Computational thinking**.

KEYWORDS

mathematics, CS+X, interdisciplinary

ACM Reference Format:
Michelle Friend, Andrew W. Swift, Betty Love, and Victor Winter. 2023. A Wolf in Lamb's Clothing: Computer Science in a Mathematics Course.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGCSE 2023, March 15–18, 2023, Toronto, ON, Canada
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9431-4/23/03…$15.00
https://doi.org/10.1145/3545945.3569736

In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE 2023), March 15–18, 2023, Toronto, ON, Canada.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3545945.3569736

1 INTRODUCTION

In a technologically advanced society, a basic understanding of computing is necessary for an informed populace. This is one of the motivations for the "CS For All" movement which is dedicated to ensuring *all* students receive high-quality computer science education. "CS For All" has largely worked to increase computing education offerings in compulsory education settings: elementary and secondary schools. However, colleges and universities can play a role in providing high quality access to computing opportunities, particularly for students whose pre-college experience did not include computer science. One approach to increasing opportunities for students with diverse interests to engage with computing at the college level is the "CS + X" approach, where computer science is combined with another discipline. However, this requires students to have sufficient interest in computing to add CS to their program of study.

This paper describes a different approach: a general education mathematics course that uses computational thinking and programming, with art as the output, to fulfill non-STEM students' general education distribution requirement. The course successfully taught students a variety of mathematical concepts and basic programming skills. As an alternative to college algebra, students found the course more personally meaningful and understood mathematics in a different way.

2 BACKGROUND

2.1 Expanding CS Education

The "CS For All" movement is dedicated to ensuring all students receive high-quality computer science education [18, 19]. Its multifaceted approach seeks to provide the largest number of opportunities for students possible - formal and informal education, standalone computer science classes and computer science integrated across the curriculum, at all levels. One element of this approach has been to focus on computational thinking, the way of thinking fundamental to computer science but applied to problems across disciplines [11, 22]. The "CS For All" movement has primarily focused on elementary and secondary school age students, as those

are the ages when all students must be in school, and therefore "All" are available to be introduced to CS.

The "CS + X" approach has largely focused on allowing computer science students to integrate their studies with other disciplines, allowing computer science to support innovative and exciting work in those areas. For example, the University of Maryland has added a computing minor they describe as a "X + CS" approach [13]. The University of Illinois has a "CS + X" program that provides the option to combine a computer science major with linguistics, crop sciences, or music among other disciplines [25]. A 2020 SIGCSE panel advocated for strongly themed introductory computing courses [17]. These approaches provide students the opportunity to develop expertise in computer science and also explicitly learn to use computational thinking and apply the tools of computer science to problems in personally-engaging disciplines, solving real-world problems.

2.2 Problems with General Education Mathematics

In many colleges and universities, college algebra has enjoyed exclusive status as the general education (gen ed) math course required for graduation. However, the utility of algebra-based math courses is increasingly being questioned, and new pathways are being designed [4, 5, 9, 12]. College algebra is an important foundation for STEM-based majors, many of whom are required to take calculus. However, for non-STEM majors, college algebra holds little utility, and they would be better served by other math pathways. Reportedly, 80% of students in algebra-heavy math courses do not need this content to be successful in future courses [10]. For example, many majors such as political science would be better served by a statistics course than learning about linear inequalities and systems of equations. A strong grounding in the computational thinking practices that underlie modern technology would be valuable for all students.

Furthermore, the problem goes far beyond the simple utility of algebra; its requirement is a barrier to graduation in many cases [14]. At the community college level, nearly 60% of students must take at least one developmental math course. Of those, only 33% complete the developmental math sequence and 20% complete a college-level math course [1]. Sadly, at some institutions, over 90% of low-income students and students of color must start in remedial courses.

These issues provide an opportunity to integrate computer science into mathematics in order to provide meaningful alternatives to the traditional algebra-based general education course. An integrated approach to mathematics can be less fragmented and more meaningful to students [8]. A computing-based mathematics course is likely to be substantially different from students' prior mathematics experiences, providing an opportunity to change students' attitudes and understanding about the nature of mathematics, and thus their interest and self-efficacy. For example, using visualizations increases students' mathematical creativity *and* learning [2, 3]. As digital technology advances, students can use computers for calculations, creating space for a deeper conceptual understanding.

3 A DIFFERENT APPROACH TO GEN ED MATH

For decades the only course approved to fulfill the gen ed math requirement at our university was algebra. The course was largely viewed by students as one to simply "get through." It did not engage students in an appreciation for the beauty of mathematics and mathematical thinking, and it did not inspire them to want to learn more mathematics. In short, most students did not like math before taking the course and didn't change their minds while in the course. Not surprisingly, very few of these students went on to take any further mathematics courses; of the 1600 students who took the course in 2015, only 32 (2%) went on to take calculus.

As an alternative to the traditional gen ed algebra course, we developed a course called *Introduction to Mathematical and Computational Thinking* (MCT), described in section 4.

This course also led to a change in the general education requirements, renamed and refocused from *mathematics* to *quantitative literacy*. Undergraduates now must "demonstrate competency in using mathematical, computational, or statistical methods to:

- Solve real-world problems
- Draw inferences based on a set of data or quantitative information
- Justify conclusions derived from quantitative information"

This change in requirements has lead to the development of multiple gen ed options including Statistics, Quantitative Literacy, and a Data Literacy and Visualization course. The Introduction to Mathematical and Computational Thinking course has remained popular with students even in the face of multiple alternatives.

4 INTRODUCTION TO MATHEMATICAL AND COMPUTATIONAL THINKING

Our goals for the MCT course were to (1) engage students; (2) change attitudes about math and STEM; (3) provide many opportunities for students to be creative; (4) teach mathematical thinking, not calculating; and (5) use active and inquiry-based learning to promote deeper understanding.

This course embraces the visual arts to introduce students to the foundational elements of mathematical and computational thinking. Visual patterns form the basis for explorations in arithmetic and geometric sequences, from which algebraic functions and corresponding functions in computer programs are reasoned. Furthermore, art is often very engaging for students, particularly those in the humanities, who may be more interested in creative expression than scientific rigor. Art is a non-threatening context, because it seems at counterpoint to the single correct answer many students have come to associate with math.

As shown in section 5, the course was initially offered prior to the Covid-19 global pandemic and has continued during the pandemic. Thus, the course was initially offered in person only, transitioned to entirely online course meetings, and is now offered in both formats. There is no evidence of different student outcomes based on whether the class is offered in person or online, thus the course will be described holistically, without attention to the mode of instructional delivery.

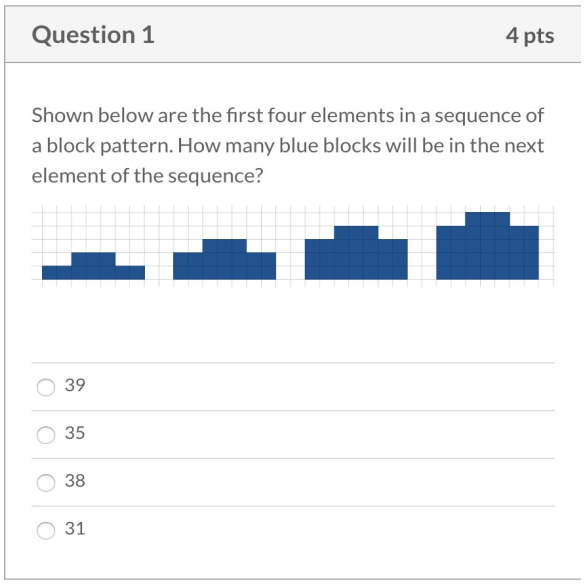


Figure 1: Sample Quiz question

4.1 Student Learning Outcomes

Mathematical learning outcomes include student proficiency in modeling two dimensional (2-D) and three dimensional (3-D) objects using algebraic and functional expressions including: (a) linear expressions and inequalities, (b) circles, spheres, polygons, and other 3-dimensional polyhedra, (c) expressions involving modular arithmetic, (d) Boolean algebra and reasoning, (e) Simple discrete mathematical reasoning, (f) Arithmetic sequences and arithmetic, functional sequences, and (g) Geometric sequences and geometric functional sequences. They apply functional reasoning tools to create 2-D and 3-D objects including: (a) coordinate transformations (shifts/offsets, dilations/scaling, flips/reflections), (b) simple discrete mathematical reasoning composition of functions, (c) multi-variable functions, and (d) simplifying (eliminating redundancy) in algebraic and function structures.

4.2 Course delivery

Each section of the course enrolls 40 students and meets twice per week for 75 minutes. Laptops are available for students to use during class, and necessary software is installed on not only these computers but also on computers available in computer labs on campus. Students are also welcome to use their own computers. There is at least one TA available in each class to support students as they work in small activities and assignments. TAs also offer frequent office hours to support students, including online office hours where students can connect virtually from home to ask questions and seek help. TAs included undergraduate and graduate math majors as well as students who had previously taken the course.

The course has been usually been taught by professors from the mathematics department, including an instructor hired specifically to support the enrollment growth in this class. It was developed as an interdisciplinary project, and a computer science professor has also taught the class at times.

4.3 Assignments

The course utilizes several types of assignments to support student learning as well as to provide formative and summative assessment.

- **Daily quizzes** are delivered through the university LMS (Canvas). These quizzes ensure that students stay engaged with the course even between meetings. Daily quizzes covered both code, such as predicting output, and purely mathematical concepts and skills such as calculating the number of pieces in a pattern at an arbitrary element. These multiple-choice quizzes are automatically graded, providing immediate formative assessment to students. An example quiz question from early in the semester is in Figure 1. Other questions asked students to interpret code by choosing the output from several options, choosing which code snippet would create an image, or identifying which element of a pattern would contain a number of blocks.
- **Homework and in-class assignments** are designed for students to practice a skill. For example, students were asked to write programs that met certain criteria, such as creating a certain image. The activities grew in complexity over the course of the semester, as students developed expertise with lower-level concepts and with the programming environment. Different instructors handled homework and in-class assignments differently. For example, some instructors had graded in-class assignments separate from homework, while others had assignments called 'homework' but gave ample time in class to work. In both cases, the goal was to ensure students the opportunity to engage with the material while in the supportive presence of the instructor and TAs.
- **Art Shows** are larger, open-ended assignments. Students are given the freedom to create artistic artifacts that met certain technical requirements. For example, one art show assignment directed "Your code must include a function with parameters for location and scale to place an object in at least three places and at least two different sizes." Art shows were graded on originality, complexity, and overall final product. Art show assignments were worth significantly more than a regular homework assignment, but also required greater complexity and allowed greater originality. Art shows also have an element of authenticity, because student work is shown in class, the "art show" element of the assignment. Further, students voted on the winners of the art show (e.g. Figures 2 and 3), and these pieces of art were shown on the course front page as well as the Mathematics Department web page about the course.
- **Exams** are summative, and assess students' ability to demonstrate skills and understanding that covered in the class. Exams asked students to write code that created specific artifacts or addressed given problems. For example, a question might prompt students to create artifacts matching the French, Mexican, and Italian flags, such as this question: "Your program must include a function called flag having parameters (xLoc,zLoc,color1,color2,color3) where (xLoc,zLoc) are the coordinates of the lower left corner of the artifact and color1 is the bottom color, color2 is the middle color, and color3 is the top color. The function call: `flag`

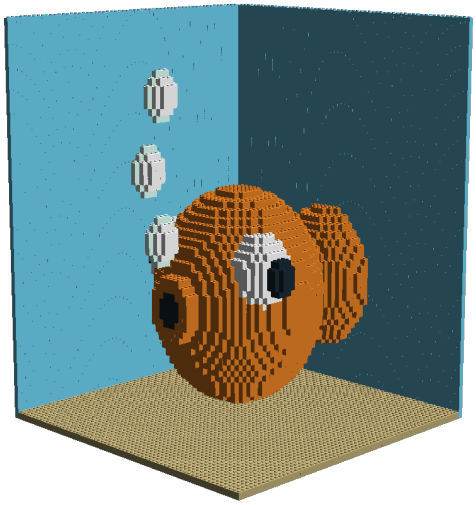


Figure 2: A winning art show entry completed in Bricklayer

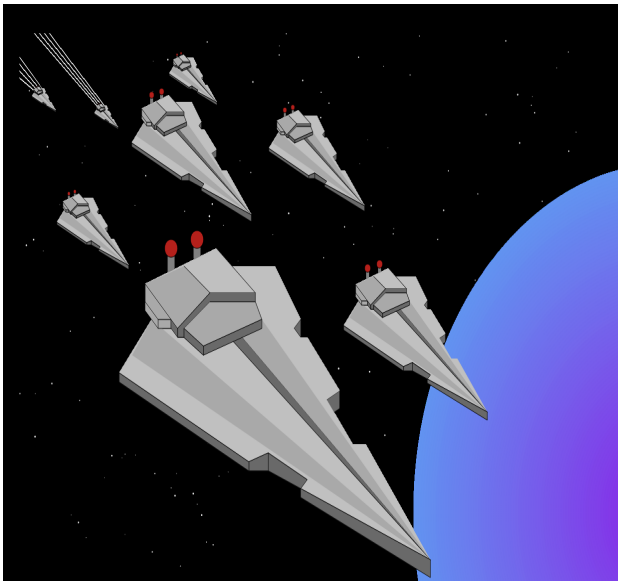
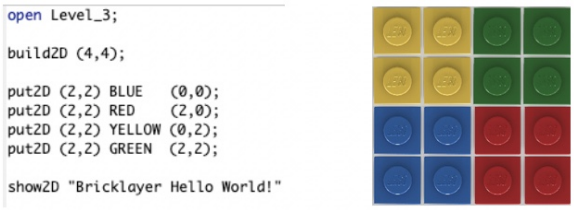


Figure 3: A winning art show entry completed in Processing

> (0,0,BLUE,WHITE,RED) would create the red, white, and blue flag above at location (0,0). Your program should make three calls to the flag function, one for each of the flags. Be precise about the size of each flag."

4.4 Programming environment

This course is considered a mathematics course, and thus the central goal was to ensure that students mastered mathematical content. Thus, it was important to select a programming environment that would support mathematical thinking.



```
open Level_3;

build2D (4,4);

put2D (2,2) BLUE   (0,0);
put2D (2,2) RED    (2,0);
put2D (2,2) YELLOW (0,2);
put2D (2,2) GREEN  (2,2);

show2D "Bricklayer Hello World!"
```

Figure 4: Sample Bricklayer Program and Output

4.4.1 Bricklayer. Initially, the course was designed to use Bricklayer [23], an open-source collection of interactive web apps, downloadable software, YouTube videos, and reading material that facilitates a thoughtful and systematic exploration of construction techniques underlying 2- and 3-dimensional block-based visual art. Such exploration provides opportunities to exercise and develop spatial reasoning abilities as well as mathematical and computational thinking skills [24]. Originally created as a tool for teaching programming and the proper way to think about programming to university freshmen CS majors, Bricklayer evolved as a "low-threshold, infinite-ceiling" system that proved effective in teaching math and coding to students as early as elementary school [7]. When executed, a Bricklayer program outputs a two- or three-dimensional block-based visual artifact that can be rendered as LEGO or in Minecraft or 3D printed. A simple Bricklayer program and its output is shown in Figure 4.

Bricklayer is a functional programming language, build on SML. Because the functions in functional programming languages mirror those in algebra, there is a closer relationship between this math and this form of programming than many others [16]. Functional programming has been shown to successfully support understanding of algebraic functions [15].

4.4.2 Blocks-first vs. Text-only. Bricklayer has both a block-based environment, "Bricklayer-lite" and a text-based interface as shown in Figure 4. In the first several offerings of the course, students were first introduced to Bricklayer-lite and taught to write programs in the block-based interface. Beginning in summer 2019, instructors discontinued use of the block interface and have students immediately begin using the text-based interface, which offers more features. Although this is at odds with prior research suggesting that block-based programming may be advantageous [20, 21], the instructors felt that students were equally successful, and that a text-first approach decreases the challenges of changing interface when the course material advances to a point beyond the capabilities of Bricklayer-lite.

4.4.3 Processing. Over time, it emerged that Bricklayer's implementation of a number of mathematical concepts were serving as impediments to student understanding of the math. For example, in math, coordinates refer to points; in Bricklayer coordinates refer to the cell. Circles in Bricklayer are approximations, and students had to disambiguate between the standard diameter formula ($2r^2$) and the Bricklayer implementation ($2r^2 + 1$). Furthermore, Bricklayer required installation of software onto each student's computer, which did not always go smoothly, particularly for students who used Apple computers.

In Fall 2020, one instructor piloted use of Processing using JavaScript. This became the standard environment the following semester. Because JavaScript is an industry-standard language and Processing is widely adopted, many learning materials already existed, meaning that the transition was smooth with little need to re-create assignments or learning materials.

5 OUTCOMES

The course was first offered in Spring 2018. 1. Table 1 shows enrollment, the software, and approach in each semester in which data was collected. In Fall 2020, one instructor piloted the use of Processing; the others used Bricklayer then transitioned in the spring semester.

5.1 Data

Quantitative data was collected via a web-based attitudinal survey. Students were asked to take the survey during the first week of the course and again during the last week of the course. Repeated measures investigated students' intention to take future math and programming courses, confidence and belonging in mathematics, and mindset about mathematics and programming.

Students also wrote "mathographies" at the beginning and end of the course. A mathography is a students' autobiography of their experience with mathematics [6]. In the pre-course mathography, students wrote about their experiences with math prior to taking the course, including their best and worst math experiences. At the end of the semester, students wrote a post-course mathography in which they wrote about their best and worst experience in the course.

Both the surveys and mathographies were treated as course assignments; students received credit for completion. Invitations to participate in data collection were issued by an education faculty who was not otherwise associated with the course, and students were assured of confidentiality. Prior to the pandemic, the education faculty visited class to invite participation, beginning in Summer 2020, invitations were issued via a recorded video embedded in the class LMS, and via e-mail from the education faculty to each student.

All data were anonomyzed using consistent ID numbers such that a single participant's data could be identified, but not the identity of that participant. Quantitative data was analyzed using R; within-subject measures examined attitudinal change from the initial survey to the end-of-course survey.

The mathographies were anonymized and imported into NVivo software by a researcher. The researcher coded a subset of the mathographies then met with the lead author to discuss and refine the codes, ensuring validity of the coding through a dependability audit. This process was repeated several times until a final set of codes was confirmed. The coding was largely in-vivo, capturing participants use of descriptive language including time-of-life (such as notable experiences happening in high school or college), and affective language, such as 'frustration' or 'proud'

5.2 Findings

5.2.1 Students have terrible prior experiences. Students' initial mathographies generally described negative experiences with mathematics, especially algebra-based mathematics. Many described prior experiences of failure, especially with the former general education college algebra class. Students generally described low-self efficacy and low interest in math, not having a sense of what it was useful for. This was echoed in the responses to the survey, where students' interest and confidence with mathematics was low.

5.2.2 Attitudes about math. The validated eight-question S-STEM instrument was used to measure student attitudes regarding mathematics. Responses were scored on a scale from 1-5 with 5 indicating more positive attitudes. These were averaged to a single score; Chronbach's alpha showed good reliability (0.91 for both pre- and post-course surveys). For the 399 students who answered both surveys, there was a significant increase in the mean response from 2.489 ($sd = 0.97$) to 2.660 ($sd = 0.99$)($W = 43228$, $p < .001$). This was a small effect ($r = .293$).

Within the S-STEM survey, one question on the survey investigated student agreement with the statement "I am good at math." For the 399 students who answered on both surveys, there was a significant increase on a five-point Likert scale from disagreement ($m = 2.489$, $sd = 1.27$) to neutrality ($m = 2.930$, $sd = 1.26$)($W = 19558$, $p < 0.001$). This was a moderate effect ($r = 0.379$). We identify this question as particularly important because beyond general attitudes towards math, it demonstrates a shift in students' identity.

5.2.3 Attitudes about programming. Six items adopted from the S-STEM instrument assessed student attitude towards programming. (E.g. "I would like to learn more about computers & programming.") Responses were scored on a scale from 1-5 with 5 indicating more positive attitudes. These were averaged to a single score; Chronbach's alpha showed good reliability (0.79 for both pre- and post-course surveys). For the 395 students who answered both surveys, there was a small but significant decrease in the mean response from 3.881 ($sd = 0.69$) to 3.699 ($sd = 0.80$) ($W = 19482$, $p < 0.001$). This was a small effect ($r = 0.252$).

5.2.4 Creativity was engaging. The ending mathographies were particularly notable in their depictions of what aspects of the class students had found useful or engaging. By far the most popular course assignments were the art shows, where students were directed to write programs which met certain requirements (such as a number of function calls) but the output was completely open-ended. Some students created algorithmically complex but visually straightforward projects, others created simple algorithms that displayed output with significant artistic merit. Students described appreciating the open-ended nature of the assignment, such as the student who wrote, "One of my peak experiences was when we did an assignment involving pixel art. We got the freedom to choose any type of art picture that we would want to create." Instructors noted that students frequently put in far more effort on art show assignments than others.

Semester	Enrollment	Software	Approach
Spring 2018	14	Bricklayer	Blocks & Text
Fall 2018	58	Bricklayer	Blocks & Text
Spring 2019	74	Bricklayer	Blocks & Text
Summer 2019	40	Bricklayer	Text only
Fall 2019	116	Bricklayer	Text only
Spring 2020	107	Bricklayer	Text only
Fall 2020	122	Bricklayer	Text only
Fall 2020	40	Processing	Text only
Spring 2021	92	Processing	Text only
Summer 2021	14	Processing	Text only
Fall 2021	129	Processing	Text only

Table 1: MCT Enrollment & Approach

6 CONCLUSION

Courses like Mathematical and Computational Thinking provide an opportunity to meet several goals: to provide an engaging and meaningful alternative to the standard college algebra course; to increase student interest, confidence, and competence with mathematics; and to expose students to computational thinking and computer science through integration with mathematics. The course was generally successful in meeting these goals: students were able to learn mathematics and write programs that could solve defined problems, as demonstrated through a variety of assessment tools including quizzes, open-ended assignments, and exams. Qualitative results suggest that students enjoyed the class and felt more successful than in the traditional algebra class. A number of the post-course mathographies explicitly mentioned that this was a new and much more appealing kind of math than their previous experiences.

There is a legitimate concern, which is supported by the evidence, that students in courses where computer science is integrated into other disciplines will not recognize that they are engaging in computational thinking or computer science, and thus some of the value to the computer science field of these integrated approaches may be lost. It is true that there is no quantitative evidence that this approach increased students' interest or confidence in computing directly. There were anecdotal suggestions that it was highly effective for some students, such as a student who took a job in technology and told the professor that she attributed applying for the job to the confidence in her abilities that the course had provided. That said, the population who enrolls in gen ed math is not a population who might be expected to pursue CS or STEM degrees and careers, so any movement towards increased knowledge, understanding, skill, and interest is of value. Additionally, there is evidence in science and math education that compulsory K-12 school experiences in those subjects diminishes interest for many students, who do not realize that the rote, "cookbook" experiences they have in school do not resemble the lived experiences of workers in those industries. This was evident in the descriptions of students' early experiences in math. In this way, "sneaking" computing experiences into other settings may increase engagement for students who would be turned off in traditional computer science courses as well as providing an alternative disciplinary experience.

This paper focused on describing the Mathematical and Computational Thinking course and providing evidence of its effectiveness with students. In the years since the course's inception, it has been taught by multiple instructors, both from math and computer science. The course has been adopted by a local community college and high schools as well. Instructors generally use active learning techniques, in which students work together and individually in class on open-ended assignments, a research-supported pedagogy gaining popularity in both mathematics and computer science. Open-ended assignments such as the art shows are engaging for students and teachers – faculty of this course disseminate student art works with pride and joy. The art shows have also proven to be "cheat-resistent": because students must create a plan for their art and create projects with personal meaning, they cannot easily copy someone else's work and pass it off as their own. Because the emphasis is on demonstrating mathematical concepts through art rather than on programming itself, it is easier to assess, for example, whether the art demonstrates a coherent use of patterns, has sufficient decomposition, or uses multi-variable functions.

Although the description of the course and findings are preliminary, the evidence collected so far suggests that this course is effective in meeting the goals, both for student learning and generally for the larger community.

ACKNOWLEDGMENTS

This work was supported by the U.S. National Science Foundation Award 1712080.

REFERENCES

[1] Thomas Bailey, Dong Wook Jeong, and Sung Woo Cho. 2010. Referral, enrollment, and completion in developmental education sequences in community colleges. *Economics of Education Review* 29, 2 (2010), 255–270. https://doi.org/10.1016/j.econedurev.2009.09.002

[2] Ali Bicer. 2021. Multiple representations and mathematical creativity. *Thinking Skills and Creativity* 42 (2021). https://doi.org/10.1016/j.tsc.2021.100960

[3] Jo Boaler, Lang Chen, Cathy Williams, and Montserrat Cordero. 2016. Seeing as Understanding: The Importance of Visual Mathematics for our Brain and Learning. *Journal of Applied Computational Mathematics* 05, 05 (2016). https://doi.org/10.4172/2168-9679.1000325

[4] David M. Bressoud. 2014. *Attracting and Retaining Students To Complete Two-and Four-Year Undergraduate Degrees in Stem: the Role of.* Technical Report. National Academies of Sciences, Engineering, and Medicine. http://sites.nationalacademies.org/cs/groups/dbassesite/documents/webpage/dbasse{_}088835.pdf

[5] Pamela Burdman. 2015. *Degrees of freedom: Varying routes to math readiness and the challenge of intersegmental alignment.* Technical Report. Policy Analysis for California Education, PACE, Stanford, CA.

[6] Corey Drake. 2006. Turning points: Using teachers' mathematics life stories to understand the implementation of mathematics education reform. *Journal of Mathematics Teacher Education* 9, 6 (2006), 579–608. https://doi.org/10.1007/s10857-006-9021-9

[7] Michelle Friend, Michael Matthews, Victor Winter, Betty Love, Deanna Moisset, and Ian Goodwin. 2018. Bricklayer: Elementary Students Learn Math Through Programming and Art. In *Proceedings of the 49th ACM Technical Symposium on Computer Science Education* (Baltimore, Maryland, USA) *(SIGCSE '18)*. ACM, New York, NY, USA, 628–633. https://doi.org/10.1145/3159450.3159515

[8] Joseph M. Furner and David D. Kumar. 2007. The mathematics and science integration argument: A stand for teacher education. *Eurasia Journal of Mathematics, Science and Technology Education* 3, 3 (2007), 185–189. https://doi.org/10.12973/ejmste/75397

[9] Amy Getz, Heather R. Ortiz, Rebecca Hartzler, and Francesca Leahy. 2016. *Modernizing Entry-Level Mathematics Programs : The Case for Mathematics Pathways.* Technical Report. The Charles A. Dana Center at The University of Texas at Austin, Austin, TX. 1–8 pages. https://dcmathpathways.org/sites/default/files/resources/2016-10/DCMPCalltoActionModernizingEntryLevelMathematicsPrograms{_}20161013.pdf

[10] Sheldon P. Gordon. 2008. What's wrong with college algebra? *Primus* 18, 6 (2008), 516–541. https://doi.org/10.1080/10511970701598752

[11] Shuchi Grover and Roy Pea. 2013. Computational Thinking in K-12: A Review of the State of the Field. *Educational Researcher* 42, 1 (feb 2013), 38–43. https://doi.org/10.3102/0013189X12463051

[12] Richard Kazis and Jenna Cullinane. 2015. *Modernizing Mathematics Pathways at Texas Universities.* Technical Report. The Charles A. Dana Center at The University of Texas at Austin, Austin, TX.

[13] Susan M. Mitchell, Anupam Joshi, and Katharine C. Cole. 2020. X + CS : A Computing Pathway for Non-Computer Science Majors. In *American Society for Engineering Education Mid-Atlantic Spring Section.* 1–10. https://par.nsf.gov/biblio/10192230

[14] Karen Saxe and Linda Braddy. 2015. *A Common Vision for Undergraduate Mathematical Sciences Programs in 2025.* The Mathematical Association of America.

[15] Emmanuel Schanzer, Kathi Fisler, Shriram Krishnamurthi, and Matthias Felleisen. 2015. Transferring Skills at Solving Word Problems from Computing to Algebra Through Bootstrap. *Proceedings of the 46th ACM Technical Symposium on Computer Science Education - SIGCSE '15* (2015), 616–621. https://doi.org/10.1145/2676723.2677238

[16] Emmanuel Tanenbaum Schanzer. 2015. *Algebraic Functions, Computer Programming, and the Challenge of Transfer.* Doctoral dissertation. Harvard Graduate School of Education. https://doi.org/10.1016/S0140-6736(09)60087-8

[17] Robert H. Sloan, Valerie Barr, Heather Bort, Mark Guzdial, Ran Libeskind-Hadas, and Richard Warner. 2020. CS + x meets cs 1: Strongly themed intro courses. In *SIGCSE 2020 - Proceedings of the 51st ACM Technical Symposium on Computer Science Education.* 960–961. https://doi.org/10.1145/3328778.3366975

[18] Carol Tate. 2022. *Computer Science for All: A Case Study in Curriculum Reform.* Ph. D. Dissertation. Rutgers, The State University of New Jersey.

[19] Carol Tate, Julie Remold, and Marie Bienkowski. 2018. Pursuing the vision of CS for all: Views from the front lines. *ACM Inroads* 9, 3 (2018), 48–52. https://doi.org/10.1145/3230704

[20] David Weintrop. 2018. Blocks or Text? How Programming Language Modality Makes a Difference in Assessing Underrepresented Populations. In *Proceedings of the International Conference on the Learning Sciences.* London, UK, 328–335. https://www.researchgate.net/publication/326131890

[21] David Weintrop and Uri Wilensky. 2017. Comparing block-based and text-based programming in high school computer science classrooms. *ACM Transactions on Computing Education* 18, 1 (oct 2017). https://doi.org/10.1145/3089799

[22] Jeannette M. Wing. 2006. Computational thinking. *Commun. ACM* 49, 3 (mar 2006), 33. https://doi.org/10.1145/1118178.1118215

[23] Victor Winter. 2015. The World Needs More Computer Science! What to do? In *Digital Media, Tools, and Approaches in Teaching and Their Added Value*, D. Conway, S. A. Hillen, M. Landis, M. T. Schlegelmilch, and P. Wolcott (Eds.). Waxmann Verlag GmbH, Germany, 119–141.

[24] Victor Winter, Betty Love, and Cindy Corritore. 2018. The art of the Wunderlich cube and the development of spatial abilities. *International Journal of Child-Computer Interaction* (2018). https://doi.org/10.1016/j.ijcci.2018.03.003

[25] Julie Wurth. 2017. UI's 'CS + X' degrees add up on many different levels. *The News-Gazette* (2017). https://www.news-gazette.com/news/ui-s-cs-x-degrees-add-up-on-many-different/article{_}7054923f-4216-5984-9869-17e595cdc2fb.html